

Kubernetes Deployment Evidence
Cloud-Based IoT Environmental Telemetry Platform
Seid Cubro - Charles Shoppel

1. Deployment Evidence:

```
PS C:\Users\seidc\cs361-iot-cloud-telemetry-platform> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
api-5c5b4969fb-vfsxv                1/1     Running   0           24s
ingestion-58fccd695f-fkjxp          1/1     Running   0           24s
PS C:\Users\seidc\cs361-iot-cloud-telemetry-platform> kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)    AGE
api-svc             ClusterIP   10.96.22.226   <none>        8082/TCP    24s
ingestion-svc       ClusterIP   10.96.239.72   <none>        8081/TCP    24s
kubernetes           ClusterIP   10.96.0.1      <none>        443/TCP     25h
PS C:\Users\seidc\cs361-iot-cloud-telemetry-platform> kubectl get ingress
NAME            CLASS    HOSTS          ADDRESS    PORTS    AGE
cs361-ingress   nginx    localhost                80       24s
PS C:\Users\seidc\cs361-iot-cloud-telemetry-platform> kubectl get hpa
NAME        REFERENCE          TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
api-hpa     Deployment/api      <unknown>/25%   1         3         1          25s
PS C:\Users\seidc\cs361-iot-cloud-telemetry-platform> kubectl top nodes
NAME                CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
cs361-control-plane 1001m        8%       921Mi           26%
```

2. Technical Explanation

This screenshot demonstrates that the IoT telemetry platform has been successfully deployed to a Kubernetes cluster (kind) and is operational with service routing and autoscaling configured. The `kubectl get pods` output confirms both application microservices, `api` and `ingestion`, are running as Kubernetes deployments with healthy containers (READY 1/1, STATUS Running). The `kubectl get svc` output shows each deployment is exposed internally through a ClusterIP Service (`api-svc` on port 8082 and `ingestion-svc` on port 8081), which provides stable service discovery and load balancing inside the cluster. The `kubectl get ingress` output verifies an NGINX Ingress resource (`cs361-ingress`) exists to route HTTP traffic based on URL paths/hostnames to the correct backend service, satisfying the ingress/service discovery requirement. The `kubectl get hpa` output confirms a Horizontal Pod Autoscaler (`api-hpa`) has been created for the API Deployment with a target CPU utilization threshold (25%) and a scaling range of 1–3 replicas, meeting the autoscaling requirement. Finally, `kubectl top nodes` proves the Kubernetes Metrics API is available (metrics-server is functioning) by returning real CPU and memory usage for the node, which is required for HPA to make scaling decisions.

Command breakdown:

- `kubectl get pods`
 - Shows the live workload state (Deployments created pods and they are healthy)
 - Evidence: both `api-*` and `ingestion-*` pods are Running and Ready (1/1).

- `kubectl get svc`
 - Shows service discovery and stable networking inside the cluster
 - Evidence: `api-svc` and `ingestion-svc` are ClusterIP services mapping traffic to the correct pods on ports 8082 and 8081
- `kubectl get ingress`
 - Shows external HTTP routing is configured through Kubernetes Ingress
 - Evidence: `cs361-ingress` exists with `nginx` class and host `localhost`, enabling path/host-based routing to services
- `kubectl get hpa`
 - Shows autoscaling configuration for the API service
 - Evidence: `api-hpa` targets `Deployment/api`, scales between 1 and 3 pods, with CPU target 25%.
- `kubectl top nodes`
 - Shows the metrics pipeline is working (`metrics-server` installed/healthy)
 - Evidence: returns real CPU/memory usage for `cs361-control-plane`, which is required for HPA-driven autoscaling